

Task 1:

EX1

```
Set<Student> students = new HashSet<>();
students.add(new Student(1, "ahmed"));
students.add(new Student(1, "ahmed"));
students.add(new Student(2, "osama"));
students.add(new Student(2, "osama"));
students.add(new Student(3, "ali"));
```

```
System.out.println(students);
```

The question is why didn't it remove the duplicated when I made overriding to .equal()?????????

Cuz I have to make overriding to both of equal method and hascode together

⚠️ ملاحظة برمجية هامة جداً (The Contract):

في الجافا، هناك قاعدة صارمة تقول: إذا قمت بعمل Overriding لـ equals () ، يجب عليك إجبارياً عمل Overriding لـ hashCode ().

إذا قمت بعمل Overriding لـ equals فقط بدون hashCode ، فإن الـ HashSet قد لا تعمل بشكل صحيح وقد تسمح بالتكرار. لأنها تفحص الـ hashCode أولاً للبحث في المجموعات (Buckets) داخل الذاكرة قبل أن تستخدم equals .

```
import java.util.Objects;
```

```
public class Student {
```

```
    int id;
```

```
    String name;
```

```
    public Student(int id, String name) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
    }
```

```
    // الذاكرة عناوين من بدلاً القيم لمقارنة: الأولى الدالة
```

```
    @Override
```

```
    public boolean equals(Object o) {
```

```
        if (this == o) return true; // فوراً true رجع ،العنوان نفس هما لو
```

```
        if (o == null || getClass() != o.getClass()) return false; // false رجع ،طالب مش أو فاضي الثاني لو
```

```
        Student student = (Student) o; // إلى Object التحويل
```

```
        // name و نفس id نفس ليهم لو متساويين الطالبين: بنقوله هنا
```

```
        return id == student.id && Objects.equals(name, student.name);
```

```
    }
```

```
    // الـ HashSet الـ في العناصر حفظ مكان لتنظيم: الثانية الدالة
```

```
    @Override
```

```
    public int hashCode() {
```

```
        return Objects.hash(id, name);
```

```
    }
```

```
    // (الذاكرة عنوان) الطلاسم بدل والرقم الاسم يظهر نطبع لما عشان: إضافية دالة
```

```
    @Override
```

```

public String toString() {
    return "Student{" + "id=" + id + ", name=" + name + "\" + '\"';
}
}

```

EX2 *

```

;Student student1 = new Student(1, "ahmed")
;Student student2 = new Student(2, "osama")
;Student student3 = new Student(1, "ali")

;()<>Set<Student> students = new HashSet
;students.add(new Student(1, "ahmed"))
;students.add(new Student(2, "osama"))

;System.out.println(students.contains(student1))
;System.out.println(students.contains(student2))
;System.out.println(students.contains(student3))

```

The Question is I made overriding to the the equal method but still return false????????????????

السبب الأول (وهو الأكثر شيوعاً): أنت لم تَقم بعمل **Overriding** لـ `hashCode`

لقد قمت بعمل **Overriding** لدالة `equals` فقط ونسيت `hashCode`.

الـ `HashSet` تعتمد على الـ `hashCode` أولاً لتحديد "الرف" (Bucket) الذي ستبحث فيه داخل الذاكرة.

- بما أنك لم تغيّر دالة `hashCode`، فإن `student1` يمتلك `Hash` مختلفاً تماماً عن `new Student(1, "ahmed")` الموجود داخل المجموعة (لأن الدالة الافتراضية تعتمد على عنوان الذاكرة).
- بالتالي، الـ `HashSet` ستبحث في "رف" خاطئ، ولن تجد العنصر، ولن تكلف نفسها عناء استخدام دالة `equals` من الأساس، فترجع `false`.

الحل: يجب أن تقوم بعمل **Overriding** لـ `hashCode` مع `equals`.

السبب الثاني: خطأ في كتابة دالة `equals` (**Overloading** بدلاً من **Overriding**)

بعض المبرمجين يكتبون دالة `equals` هكذا:

```

import java.util.Objects

} public class Student
;int id
;String name

} public Student(int id, String name)

```

```
;this.id = id
;this.name = name
{
```

```
// 1. التغلب على السبب الثاني (التأكد من استخدام Object)
@Override
} public boolean equals(Object o)
;if (this == o) return true
;if (o == null || getClass() != o.getClass()) return false
```

```
;Student student = (Student) o
;return id == student.id && Objects.equals(name, student.name)
{
```

```
// 2. التغلب على السبب الأول (إجبار الجافا على إعطاء نفس الـ Hash لنفس البيانات)
@Override
} public int hashCode
;return Objects.hash(id, name)
{
{
```

EX3

```
;Student student1 = new Student(1, "ahmed")
;Student student2 = new Student(2, "osama")
;Student student3 = new Student(1, "ali")

;()<>HashMap<Student, Student> map = new HashMap
;map.put(new Student(1, "ahmed"), new Student(1, "ahmed"))
;map.put(new Student(2, "osama"), new Student(2, "osama"))

;System.out.println(map.containsKey(student1))
;System.out.println(map.containsKey(student2))
;System.out.println(map.containsKey(student3))

;System.out.println(map.containsValue(student1))
;System.out.println(map.containsValue(student2))
;System.out.println(map.containsValue(student3))
```

The question is why MAP when I made overriding it worked out not like the set

السّر هو: الـ *HashSet* في الجافا هي في الأساس عبارة عن *HashMap* متخفية!

لو فتحت السورس كود (*Source code*) بتاع الجافا نفسه وبصيت على برمجة الـ *HashSet*، هتلاقيها من جوه بتستخدم *HashMap* عشان تخزن العناصر (بتخزن العنصر كـ *Key*، وبتحط *Value* وهمية فاضية).

بما أن الاثنين من جوه شغالين بنفس الميكانيزم، فمستحيل الـ `HashMap` تشتغل معاك صح وتطلع `true`، وفي نفس الوقت الـ `HashSet` تطلع `false` لو بتستخدم نفس كلاس الـ `Student` بالظبط.

طيب ليه الـ `HashMap` اشتغلت معاك صح المرة دي؟

التفسير الوحيد المنطقي هو أنك وأنت بتجرب كود الـ `HashMap` (في EX3)، أنت كنت كاتب الـ `Overriding` في كلاس الـ `Student` بشكل صحيح `100%` (يعني عامل `Override` لـ `equals` وبتاخذ `Object`، وعامل `Override` لـ `hashCode`).

لو أخذت نفس كلاس الـ `Student` اللي اشتغل معاك في الـ `HashMap` ده، ورجعت جربته تاني على كود الـ `HashSet` (في EX2)، أضمن لك مليون في المية إن الـ `Set` هي كمان هتطلع `true` ومش هتديك `false` أبداً.

خلينا نشرح إيه اللي حصل في كود الـ `HashMap` (EX3):

الـ `HashMap` متقسمة لجزئين: `Key` (مفتاح) و `Value` (قيمة).

1. بالنسبة لـ `containsKey` (البحث في المفاتيح):

• بتشتغل بنفس طريقة الـ `Set` بالظبط.

• أول حاجة بتجيب الـ `hashCode` () بتاع `student1` عشان تعرف هو متخزن في أي "رف" في الذاكرة.

• وبعدين تستخدم دالة `equals` () عشان تتأكد إن البيانات متطابقة (1 و "ahmed").

• بما أنك عامل `Overriding` للاتنين صح، هتلاقيه وهرجع `true`.

2. بالنسبة لـ `containsValue` (البحث في القيم):

• هنا الـ `HashMap` لا تستخدم الـ `hashCode` نهائياً.

• هي بتلف على كل القيم (`Values`) الموجودة جواها واحدة واحدة، وتستخدم دالة `equals` () فقط عشان تقارن هل القيمة دي بتساوي `student1` ولا لا.

-
-
-

بما إن دالة `equals` عندك مضبوطة، هتقارن `student1` بالقيمة اللي متخزنة وتلاقي الـ `id` والـ `name` متطابقين، فهترجع `true`.

الخلاصة:

الـ `HashMap` والـ `HashSet` وجهان لعملة واحدة في البحث (لأنهم بيعتمدوا على الـ `Hash` الـ `Table`). لو الكلاس مكتوب صح، الاتنين هيشغلوا ويدوك `true`. لو مكتوب غلط، الاتنين هيدوك `false`.

سبب أن الـ `Set` طلعت `false` معاك قبل كدة إن كلاس `Student` وقتها كان ناقصه `hashCode` أو مكتوب فيه `equals` بطريقة `Overloading` بالغلط، ولما جربت الـ `Map` كنت عدلت الكلاس لنسخة صحيحة.